GOLD: Arrays (Mon Sep 14)

Interactive Menus (from Friday)

Interactive menus: keep asking until "quit"

Lots of programs don't run once and stop.

Menus keep offering choices until you're done.

Think of a game menu, an ATM, or an ordering kiosk.

That pattern combines two tools you already have:

- a **`do-while`** loop : show the menu **at least once**, and repeat until the user quits
- an **arrow `switch`** : run the right action for whatever they chose.

The menu-loop pattern

```
void main() {
    String choice;
    do {
        // 1. print the menu of options
        choice = IO.readln("Choose: ");
        // 2. act on the choice
        switch (choice) {
            case "1" -> IO.println("option 1 chosen");
            case "2" -> IO.println("option 2 chosen");
            case "q" -> IO.println("Goodbye!");
            default  -> IO.println("Not a valid choice.");
        }
    } while (!choice.equals("q"));
}
```

- **`do-while`** shows the menu before getting/checking input
    - the user is prompted at least once

- **`switch`** sends each choice to its own action.
    - No need for parsing or `equals`
- **`default`** catches typos instead of crashing or ignoring them
- The loop ends when the choice is the sentinel

Motivating Arrays

Too many temperatures

Suppose you're logging the temperature once an hour for a day. With the tools you have now, you'd write:

```
double temp0, temp1, temp2, temp3, temp4, temp5, temp6, temp7,
       temp8, temp9, temp10, temp11, temp12, temp13, temp14,
       temp15, temp16, temp17, temp18, temp19, temp20, temp21,
       temp22, temp23;
```

How would you compute the average? It wouldn't be fun.

Want the data for a whole *week*? 168 names.

One name, many values

Mathematicians write subscripted variables: t_0, t_1, t_2, ..., t_{23}.

Java writes:

```
double[] temp = new double[24];
```

That single line creates twenty-four `double` slots, named:

```
temp[0], temp[1], temp[2], …, temp[23]
```

The number in brackets is the **index** (the subscript).

Because the index can be a *variable*, you can now reach any slot in a loop:

```
for (int i = 0; i < 24; i++)
    temp[i] = readSensor(i);     // touch all 24 with three lines
```

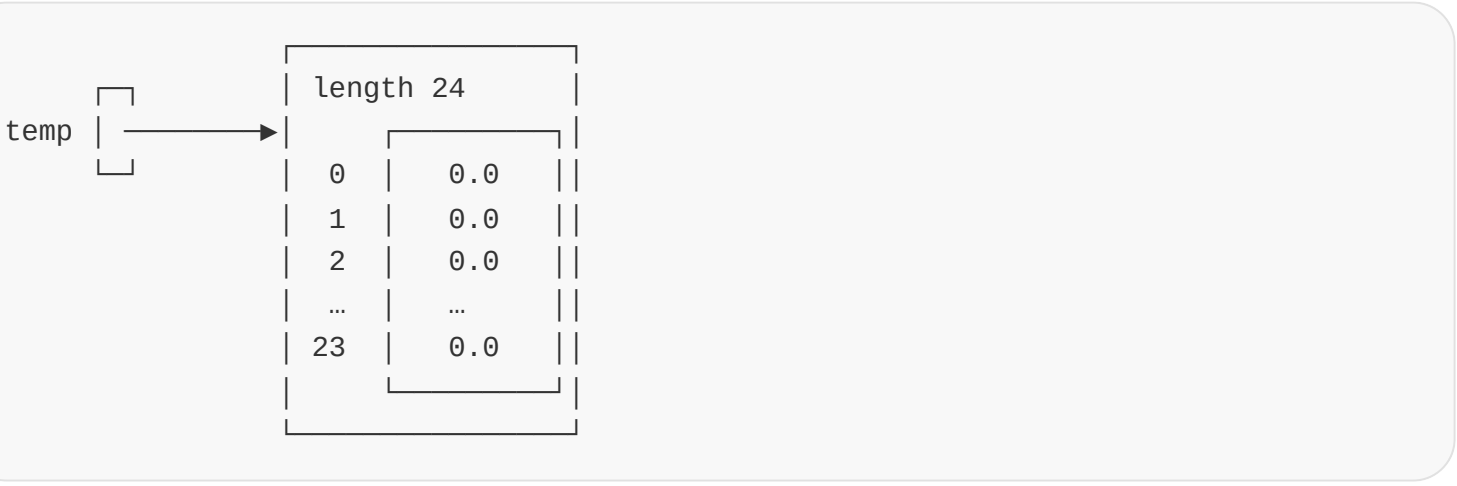
Array Picture

A picture of an array

`double[] temp = new double[24]`

builds an **array object** out in memory and points `temp` at it.

Every slot starts at the default for its type. For `double`, that's `0.0`:



What's important here?

- the array knows its own **length** (24),
- `temp` is a reference: it holds the location of the array in memory, which we draw as an arrow to the array, not the array itself.

Arrays are the second **reference type** that we've met so far.

Why Arrays?

Why? Data structures

- A single variable holds a single value.

- Real programs need to handle data **in bulk**
    - a list of temperatures,
    - a roster of students,
    - a deck of cards

- and must treat the whole collection as *one single thing* you can pass around, loop over, and return.

This bulk holding/processing of data is known as a **data structure** (also called an *aggregate data type*).

The array is the simplest one.

Describing Data Structures

We describe any data structure by three questions:

- **Contents:** are the values all
    - the same type (*homogeneous*)
    - or mixed types (*heterogeneous*)?

- **Size:**
    - fixed (*static*)
    - or can it grow and shrink (*dynamic*)?

- **Access:** To find a piece of data,
    - can you jump straight to any element (*random access*),
    - or must you walk through them in order (*sequential access*)?

Classifying Arrays

In Java, an array is:

Question	Array's answer
Contents	**homogeneous:** every element is the same declared type
Size	**static:** fixed the moment you create it; it never grows
Access	**random:** `a[i]` jumps straight to element `i`, instantly
Identity	always a **reference type** (a "container object")

Analogy: A `String` behaves like an array of characters:

- fixed length,
- every position the same type (`char`),
- you reach any position with `charAt(i)`.

Array Declaration

Three operations: declare, create, access

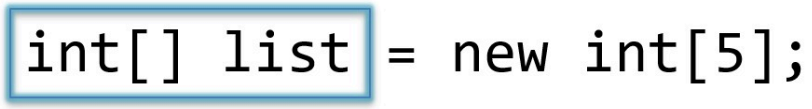
Everything you do with arrays is built from three steps:

```
int[] list;                 // 1. DECLARE  "list will refer to an int array"
list = new int[5];          // 2. CREATE   make the 5-slot array object
list[3] = 42;               // 3. ACCESS   read or write one element
```

Usually you declare and create on one line:

```
int[] list = new int[5];
var students = new String[10];     // 'var' works too because type is inferred
```

- **Declare:**
  - In general `type[] name` where the `type` is any primitive or reference type.



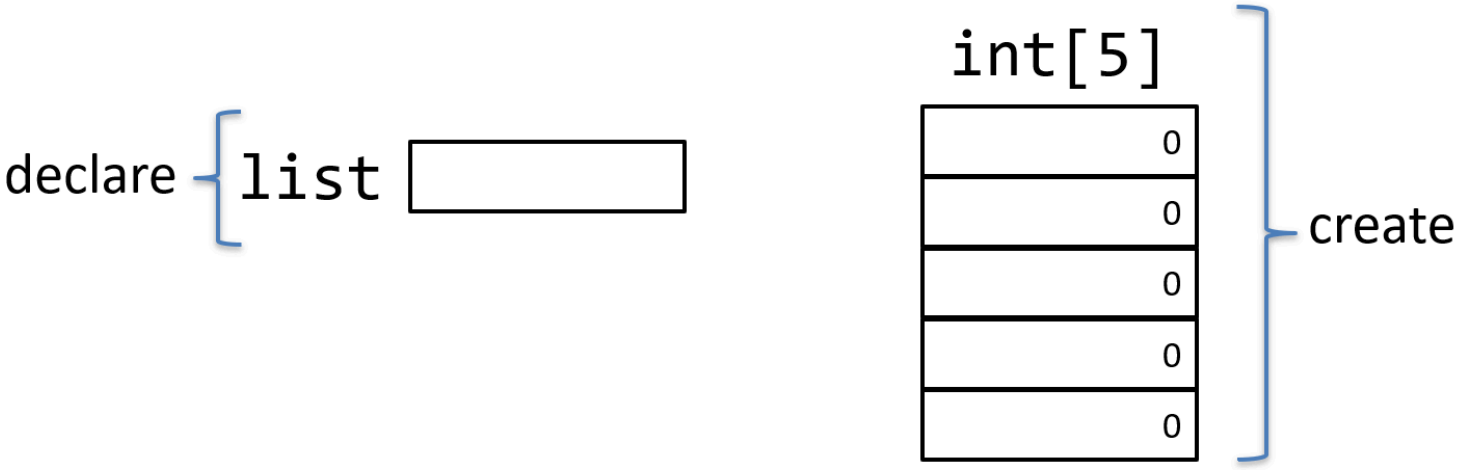


  - The declaration `int[] list;` alone creates a box that can hold an array reference, but does not fill it.

  - The default value is `null` but Java will not let us use the value of an uninitialized variable.

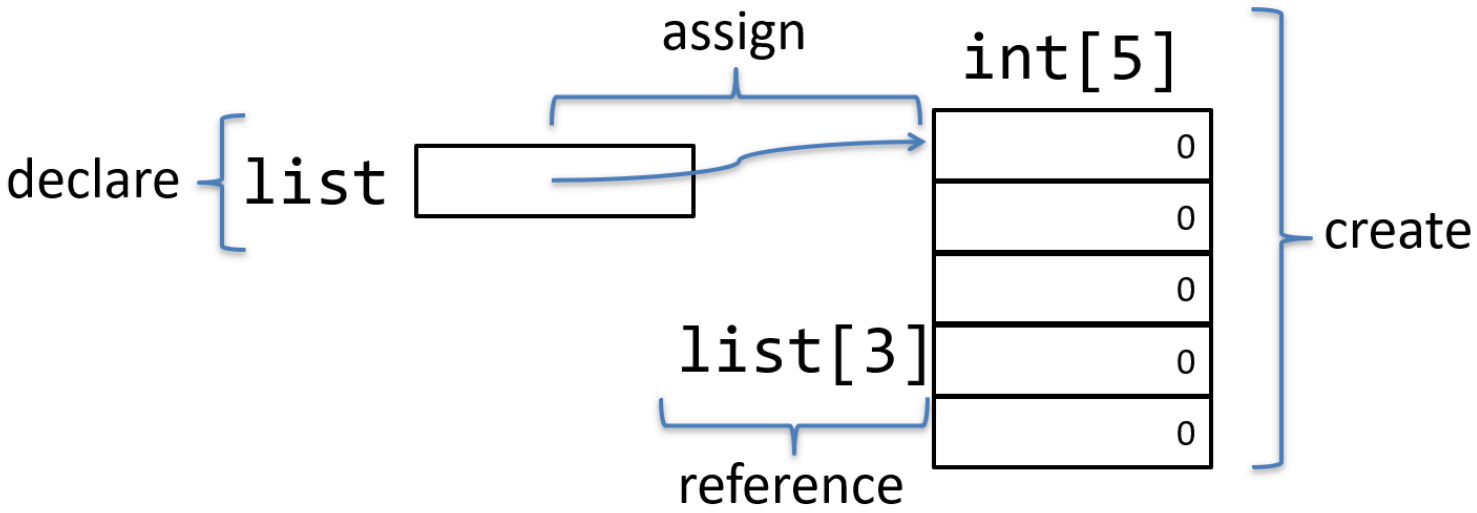
- **Create:**
  - We write `new type[size]` where *size* is any `int` expression.

```
int[] list = new int[5];
```



- **Access:**
  - *name*`[`*int*`]` where the index is any `int` -valued expression.

```
int[] list = new int[5];
```



Important Note:

For a `String[]`, the slots are **references**, so they default to `null`, not `0`:

```
String[] names = new String[5];   // every slot starts as null
```

This is one way we can get `NullPointerException` by accident.

Pause and predict

```
String[] names = new String[3];
IO.println(names[0]);
```

What prints?

- **A.** Prints an empty line (`""`)
- **B.** Prints `null`
- **C.** Crash: `NullPointerException`
- **D.** Compile error

Array Bounds

Predict: defaults and out-of-bounds

```
int[] list = new int[5];
IO.println(list[0]);        // (a) what prints?
IO.println(list[4]);        // (b) what prints?
IO.println(list[5]);        // (c) what happens?
```

Pause and predict

Q. What happens at **(c)**?

- **A.** Prints `0`, the default value
- **B.** Prints garbage from memory
- **C.** Crash: `ArrayIndexOutOfBoundsException`
- **D.** Compile error

Looping over an array

Use the `length` field plus the counting `for` loop:

```
void main() {
    int[] scores = {88, 92, 75, 100, 64};  // this is how we specify an array literal

    int sum = 0;
    for (int i = 0; i < scores.length; i++)
        sum += scores[i];

    double average = (double) sum / scores.length;
    IO.println(String.format("average = %.1f", average));   // average = 83.8
}
```

`for (int i = 0; i < a.length; i++)` is *the* array idiom.

We start at 0, stop **before** `length`.

Compile-time Initialization

The `{...}` initializer

If you know the contents of your array up front, skip `new` and just list them in braces.

- These are known as **array literals** or **initializers**.
- Java counts them and sizes the array for you:

```
int[]    days   = {31,28,31,30,31,30,31,31,30,31,30,31};   // length 12
char[]   vowels = {'a','e','i','o','u'};                   // length 5
String[] suits  = {"Hearts","Spades","Diamonds","Clubs"};  // length 4
```

Otherwise, `new` fills every slot with the type's **default**:

Element type	Default
`int`, `long`, `short`, `byte`	`0`
`double`, `float`	`0.0`
`boolean`	`false`
`char`	`'\u0000'` (the null character)
any reference type (`String`, ...)	`null`

Pause and predict

```
int[] days = {31,28,31,30,31,30,31,31,30,31,30,31};
int total = 0;
for (int i = 0; i <= days.length; i++) {
    total += days[i];
}
IO.println("total = " + total);
```

What happens?

- **A.** Prints `total = 365`
- **B.** Prints `total = 366`

- **C.** Crash: `ArrayIndexOutOfBoundsException: Index 12`
- **D.** Compile error

Array Applications

The histogram pattern: an array of counters

Powerful trick: use an array where **the index carries meaning**.

To count how many words have each length `k`, make `count[k]` hold "how many words of length `k`":

```
int[] computeHistogram(String[] words) {
    int[] count = new int[50];                  // lengths 0..49
    for (int i = 0; i < words.length; i++)
        count[ words[i].length() ]++;           // bump the bucket for this length
    return count;
}
```

Key idea: We can use complex expressions inside the `[ ]` brackets for an index,
e.g. `count[words[i].length()]++`

This means: "find this word's length, and add one to that bucket."

Trace the histogram

```
String[] words = {"It", "was", "a", "dark", "and", "stormy", "night"};
int[] count = computeHistogram(words);
```

Practice: fill in the non-zero buckets of `count`:

```
count[1] = ____   count[2] = ____   count[3] = ____
count[4] = ____   count[5] = ____   count[6] = ____
```

⚠ **Warning:** We have to choose the array size in advance, so a 60-letter word would ask for `count[60]` on a 50-slot array and crash.

"Size 50 and hope" is not a great engineering decision; we'll revisit later.

iClicker Poll:

```
String[] words = {"It", "was", "a", "dark", "and", "stormy", "night"};
int[] count = computeHistogram(words);
```

After the call, what is `count[3]`?

- **A.** `1`
- **B.** `2`
- **C.** `3`
- **D.** `0`

Problem: Word List

Problem: Word List

Problem: We want to compute something about a **lot** of text, such as every word of *War and Peace* (about 566,000).

Today: how many words of each length?

Step 1: Data structures

What information should be stored in our program?

- words: an array to contain all of the words
- size: the number of actual words in the array

We have two pieces of information needed by more than one method. Passing two variables around is hard so let's create **fields.**

Fields are declared of outside any method and shared by all of them:

- `String[] words` : a big array holding the words
- `int size` : how many slots are actually filled (the array is bigger than the book)

(Pro tip: When we define classes next week, fields will go *outside* of our methods, but *inside* the class)

How big? We have to **guess:** 600,000 and hope. We'll have a better answer next time!

Step 2 : read the words

We will assume the book is being is being transmitted over standard input, as if it were being typed in by the user.

`IO.readln()` hands back `null` when the input runs out.

Each line is split into words with `line.split(" ")` . This is a String method that returns a `String[]` .

Step 3 : `int[] computeHistogram()`

We saw this one already:

- allocate an array of 50 counters; one per possible word length
- loop through all `size` words
- `histogram[words[i].length()]++`
- return the array

Next we'll look at the finished program and run it.

Demo: Word List

Problem: We want to compute something about a **lot** of text, such as every word of *War and Peace* (about 566,000).

Step 1: Data structures

Fields are declared of outside any method and shared by all of them:

- `String[] words` : a big array holding the words
- `int size` : how many slots are actually filled (the array is bigger than the book)

Step 2 : read the words

`IO.readln()` hands back `null` when the input runs out.

Each line is split into words with `line.split(" ")` .

Step 3 : `int[] computeHistogram()`

We saw this one already:

- allocate an array of 50 counters
- loop through all `size` words and compute `histogram[words[i].length()]++`
- return the array

Press **Run:** the run command feeds the book in as standard input.

What happens to the histogram if you change `MAXHIST` to 10?

Take Home: Letter Counts

Today's big idea was the **histogram pattern**: an array where the index carries meaning.

Take home challenge: Now, count the number of times each letter appears.

Implement:

`int[] letterCounts(String s)`

that returns an array of exactly 26 counts: `counts[0]` is how many times `a` appears in `s`, `counts[1]` how many `b`s, ... `counts[25]` how many `z` s.

- Capital and lowercase count the same
- Anything that isn't a letter (spaces, punctuation, digits) is ignored.

The trick: a letter's bucket is `c - 'a'` (char arithmetic). `'a' - 'a'` is 0, `'b' - 'a'` is 1, and so on.

Press **Test** to run the tests.

Boundaries to think about: the empty string (the loop must run zero times and every bucket stays 0), and `'z'`: does it land in bucket 25, or off the end?